CCEE Big Data Technologies – Comprehensive Cheatsheet

Sessions 27–33: Data Preprocessing, EDA, Kafka, Spark Streaming, Spark SQL, Spark MLlib

---

Session 27: Data Preprocessing & EDA

Data Preprocessing: Core Idea

- **Definition**: Cleaning and preparation of raw data into a clean, consistent form suitable for analysis and machine learning.
- **Goal**: Improve data quality, make data consistent, reduce size/complexity.

Key Steps in Data Preprocessing

1. **Data Cleaning**
   - Handle missing values: delete rows/columns, mean/median/mode imputation, "unknown" category.
   - Fix incorrect values, remove duplicates.
   - Detect and treat outliers.
2. **Data Integration**
   - Combine multiple sources (databases, files, APIs, logs) into one coherent dataset.
   - Resolve schema differences and key conflicts.
3. **Data Transformation**
   - Scaling/Normalization: min–max, standardization (z-score).
   - Encoding categorical variables: one-hot, label encoding.
   - Feature engineering: creating new features, log/√ transforms.
4. **Data Reduction**
   - Feature selection: remove irrelevant/redundant features.
   - Dimensionality reduction: PCA, etc.
5. **Data Discretization & Formatting**
   - Convert continuous to categorical bins (low/medium/high).
   - Ensure consistent types (int/float/string, date formats).

EDA (Exploratory Data Analysis): What & Why

- **Definition**: Preliminary systematic analysis to summarize main characteristics of data using statistics and visualizations.
- **Objectives**:
   - Understand distributions (center, spread, shape, skewness).
   - Detect data quality issues (missing values, outliers, inconsistencies).
   - Discover relationships between variables (correlation, trends, groups).
   - Guide feature engineering and model choice.

Typical EDA Workflow

1. Understand the problem and data.
2. Import and inspect data (shape, columns, types, samples).
3. Handle missing values and basic cleaning.
4. **Univariate EDA**: Histograms, boxplots, summary stats for individual variables.
5. **Bivariate/Multivariate EDA**: Scatter plots, correlation matrix/heatmap, grouped summaries.
6. Detect outliers and anomalies (boxplots, z-scores, distribution tails).
7. Summarize insights (key patterns, data issues, hypotheses for modeling).

Relationship: Preprocessing vs EDA

- Preprocessing = making data **usable** (quality and structure).
- EDA = **understanding** that data.
- In practice: iterative (initial EDA → more cleaning → more EDA).

Session 28–29: Kafka & Spark Streaming

Apache Kafka: Introduction

- **Definition**: Distributed event-streaming platform for publishing, storing, and consuming streams of records in real time.
- **Core Entities**:
  - **Topic**: Named stream of records (like a "channel").
  - **Partition**: Ordered, immutable sequence of records within a topic; enables parallelism.
  - **Producer**: Application that publishes messages to topics.
  - **Consumer**: Application that reads messages from topics.
  - **Broker**: Kafka server that stores topic partitions.
- **Main Uses**: Real-time data pipelines, log aggregation, microservices communication, real-time analytics.

Working with Kafka using Spark

- Kafka acts as the **source** (input stream); Spark reads data from Kafka topics.
- Spark **processes** data: parsing, aggregations, joins, ML models, windowing, etc.
- Spark writes results to **sinks**: Kafka topics, filesystems (HDFS/S3), databases, dashboards, REST services.
- Common options: `format("kafka")`, `option("kafka.bootstrap.servers", ...)`, `option("subscribe", "topicName")`.

Spark Streaming Architecture

- **Micro-batch Processing**: Incoming data divided into small time-based batches (e.g., every 1 sec).
- **DStream**: Discretized Stream = sequence of RDDs over time.
- **Receivers**: Ingest data from sources (like Kafka) and buffer in memory.
- **Pros**: Leverages Spark's batch engine, fault-tolerant, scalable.
- **Cons**: Latency ≥ batch interval (not per-record).

Spark Streaming vs Structured Streaming

Feature	Spark Streaming	Structured Streaming
Abstraction	DStream (RDD-based)	DataFrame/Dataset-based
API	map, flatMap, filter, reduceByKey	Standard DataFrame ops, SQL
Recommendation	Older, legacy	Modern, recommended

Structured Streaming APIs

- **Source**: spark.readStream.format("kafka" | "socket" | "files" ...)
- **Transform**: Standard DataFrame ops (select, groupBy, agg, join) and SQL.
- **Sink**: writeStream with modes (append/complete/update).
- **Features**: Checkpointing, incremental processing, watermarks (automatic).

Building Stream Processing Applications

- Define source (Kafka topic, servers, topic name, offsets).
- Parse and transform data (JSON parsing, filtering, aggregations, joins).
- Apply windowing (tumbling/sliding windows over event time).
- Handle state and fault tolerance (checkpoint location, stateful ops).
- Define sink (Kafka, DB, files, REST, custom sink).

Typical Use Cases: Real-time metrics dashboards, fraud detection, IoT sensor monitoring, clickstream analytics.

Session 30: Kafka Producer & Consumer, Kafka Connect

Kafka Producer: Setup & Config

- **Purpose**: Sends records (key–value pairs) to Kafka topics.
- **Key Configs**:
  - bootstrap.servers: List of broker addresses.
  - key.serializer, value.serializer: Convert objects to bytes.
  - acks: Delivery guarantees.
    - acks=0: Fire-and-forget, lowest latency, risk of data loss.
    - acks=1: Leader only must ack.
    - acks=all (or -1): All in-sync replicas must ack; safest for durability.
  - enable.idempotence=true: Idempotent producer avoids duplicates during retries (producer ID + sequence numbers).

Kafka Consumer: Setup & Config

- **Purpose**: Reads records from topic partitions; tracks **offset** (position).
- **Key Concepts**:
  - **Consumer Group** (`group.id`):
    - Partitions of a topic are shared across consumers in the same group.
    - Each partition processed by exactly one consumer in a group at a time.
  - **Offset**: Index of message in partition; committing offset = recording "processed safely up to here."
  - **Auto Offset Reset** (`auto.offset.reset`):
    - `earliest`: Start from beginning if no valid offset.
    - `latest`: Start from new messages.
- **Key Configs**:
  - `bootstrap.servers`, `group.id`, `key.deserializer`, `value.deserializer`.
  - `auto.offset.reset`: Where to start reading if no offset found.

Consumer Groups: Parallelism & Fault Tolerance

- **Parallelism**:
  - Topic split into partitions; partitions divided across consumers in group.
  - Each partition read by at most one consumer → no duplicate work.
  - Multiple consumers process different partitions simultaneously → increased throughput.
  - Example: 6 partitions, 3 consumers → each gets ~2 partitions, process in parallel.
- **Fault Tolerance**:
  - Group coordinated so that if one consumer dies, its partitions are **reassigned** to remaining consumers.
  - Offsets stored (e.g., in Kafka) → new/surviving consumer continues from last committed position, not from beginning.
  - Ensures messages not lost; at worst, some reprocessed (depends on commit strategy, at-least-once semantics).

Kafka Connect API

- **Definition**: Framework to move large volumes of data between Kafka and external systems without custom producer/consumer code.
- **Setup**: Usually run as separate process/cluster; uses **connectors** defined mostly by configuration (JSON/properties).
- **Two Types of Connectors**:
  - **Source Connector**: Pulls data from external systems into Kafka topics (e.g., MySQL → Kafka).
  - **Sink Connector**: Pushes data from Kafka topics to external systems (e.g., Kafka → HDFS/Elasticsearch/S3).
- **Ensures**: Scalability (workers, tasks), fault tolerance, offset management.

Session 31: Spark SQL

Spark SQL: Core Idea

- **Definition**: Spark's module for working with structured and semi-structured data using SQL queries, DataFrames, and Datasets.
- **Key Feature**: Bridges traditional relational processing (tables, schemas, SQL) with Spark's distributed computation.
- **Advantages**:
  - Integrated: Mix SQL queries with Spark code (Python/Scala/Java/R).
  - Unified data access: Read/write from Hive, JSON, Parquet, ORC, Avro, CSV, JDBC, etc.
  - Optimization: Catalyst optimizer (reordering joins, predicate pushdown) + Tungsten engine (code generation, memory management).
  - Scalability & fault tolerance: Execute in parallel across cluster.

Key Abstractions

- **DataFrame**: Distributed table with named columns; similar to DB table or Pandas DataFrame.
- **Dataset**: Strongly-typed collection of objects (mainly Scala/Java); compile-time type safety.
- **Relationship in Spark 2+**: DataFrame is essentially Dataset[Row].

Typical Usage Pattern

1. Create SparkSession.
2. Load data as DataFrame (from files, Hive, JDBC, etc.).
3. Register DataFrame as temporary view or global view.
4. Run SQL using `spark.sql("SELECT ...")` or use DataFrame operations (`select`, `filter`, `groupBy`, `agg`, etc.).

RDD vs DataFrame/Dataset

Feature	RDD	DataFrame/Dataset
Level	Low-level	High-level
Schema	Untyped	Schema-aware
Optimization	None	Catalyst optimizer
Performance	Slower	Faster

Session 32–33: Spark MLlib & Predictive Analysis

Spark MLlib: Definition

- **Definition**: Spark's distributed machine-learning library; runs ML algorithms over RDDs/DataFrames across a cluster.
- **Scalability**: Makes training and prediction scalable for big data.

MLlib Algorithms

- **Classification & Regression**: Logistic Regression, Linear Regression, Decision Trees, Random Forests, Gradient Boosted Trees (GBT).
- **Clustering**: K-means.
- **Recommendations**: ALS (Alternating Least Squares).
- **Dimensionality Reduction**: PCA, SVD.
- **Statistics, Feature Extraction & Transformation**.

MLlib APIs: spark.mllib vs spark.ml

Aspect	spark.mllib	spark.ml
Base	RDD-based	DataFrame-based
Status	Older, legacy	Newer, recommended
API	Low-level transformations	High-level Pipelines
Use	Gradually superseded	Modern standard

Spark ML Pipelines (spark.ml)

- **Transformer**: Takes a DataFrame, returns another (e.g., VectorAssembler, Scaler, trained Model).
- **Estimator**: Learning algorithm that fits a model from data (e.g., LogisticRegression).
- **Pipeline**: Sequence of stages (Transformers + Estimators).
- **PipelineModel**: Fitted pipeline used for prediction.

Predictive Analysis: Definition & Workflow

- **Definition**: Using historical data + ML models to predict unknown or future values.
- **Examples**:
  - Predict if transaction is fraudulent (classification).
  - Predict house price (regression).

Typical Predictive Modeling Workflow in Spark MLlib

1. **Data Loading**: Load data into DataFrame from files/tables.
2. **Feature Engineering & Preprocessing**:
   - Use transformers: StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler, etc.
3. **Model Training**:

    - **Classification**: LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, GBTClassifier.
    - **Regression**: LinearRegression, RandomForestRegressor, GBTRegressor.
4. **Evaluation**:
    - Evaluators: BinaryClassificationEvaluator, MulticlassClassificationEvaluator, RegressionEvaluator.
    - Metrics: Accuracy, F1, RMSE, AUC, etc.
5. **Tuning** (optional):
    - Hyperparameter tuning: CrossValidator, TrainValidationSplit, ParamGridBuilder.
6. **Prediction**:
    - Apply `model.transform()` to new/unseen data → prediction and probability columns.

Big Data Predictive Use Cases

- Clickstream behavior → predict conversion/churn (classification).
- Sensor/IoT data → predict failures (regression/classification).
- Recommendation systems → ALS collaborative filtering.
- Text data → TF-IDF + Logistic Regression classification.

Key MCQ Tips & Terminology

Term	Definition/Context
Preprocessing	Cleaning & structuring raw data; includes cleaning, integration, transformation, reduction.
EDA	Exploratory analysis using stats & visualizations to understand data patterns & quality.
Outliers	Extremely large/small values distorting mean & std deviation.
Discretization	Converting continuous variables into categorical bins (e.g., age → age groups).
Kafka Partition	Ordered, immutable log within a topic; enables parallelism.
DStream	Discretized Stream in Spark Streaming; sequence of RDDs over time.
Micro-batch	Small time-based batch of data processed together in Spark Streaming.
Consumer Group	Set of consumers sharing topic partitions; each partition read by ≤1 consumer in group.
Offset	Position/index of message in a partition; consumers commit offsets to track progress.
Idempotent Producer	Kafka producer avoiding duplicates during retries using producer ID & sequence numbers.
Source Connector	Kafka Connect type pulling data from external systems into Kafka.
Sink Connector	Kafka Connect type pushing data from Kafka into external systems.
Catalyst	Spark SQL query optimizer; performs logical and physical plan optimization.
Tungsten	Spark SQL execution engine; code generation & memory management for speed.
Transformer	Spark ML component taking DataFrame, returning another (incl. trained models).

Term	Definition/Context
Estimator	Spark ML learning algorithm fitting a model from data.
Pipeline	Sequence of Transformers & Estimators in Spark ML.
Predictive Analysis	Using historical data & ML models to predict unknown/future values.

Quick Reference: When to Use What

Technology	When to Use
Data Preprocessing	Before any analysis/modeling to ensure data quality.
EDA	Early stage; understand data before modeling.
Kafka	Real-time, high-throughput event streaming; multiple producers/consumers.
Spark Streaming	Real-time processing on Kafka/socket/file streams; micro-batch approach.
Spark Structured Streaming	Modern approach; use DataFrame APIs for streaming; recommended over DStream.
Kafka Connect	Move large data volumes between Kafka & external systems without custom code.
Spark SQL	Structured data analytics; SQL queries + distributed processing.
Spark MLlib	Scalable machine learning on distributed data; classification, regression, clustering.
Predictive Analysis	Predict future/unknown outcomes using trained ML models.

Big Data Pipeline Flow

Raw Data (CSV, JSON, Databases)
↓
Data Preprocessing (Cleaning, Integration, Transformation, Reduction)
↓
EDA (Visualization, Statistics, Pattern Discovery)
↓
Kafka (Real-time Streaming) ← Spark Structured Streaming (Processing)
↓
Spark SQL (Structured Data Analysis)
↓
Spark MLlib (Predictive Models: Classification, Regression, Clustering)
↓
Predictions & Insights

Important Exam Reminders

- **Preprocessing ≠ EDA**: Preprocessing makes data usable; EDA makes it understandable.
- **DStream vs DataFrame**: Classic Spark Streaming uses DStream (RDD-based); Structured Streaming uses DataFrame (modern, preferred).
- **Consumer Groups**: Multiple consumers in same group share partitions for parallelism; each partition read by ≤1 consumer; independent from other groups.
- **Producer vs Consumer**: Producer sends messages; consumer reads + tracks offset; they use different configs (acks vs auto.offset.reset).
- **Source vs Sink Connectors**: Source pulls external data into Kafka; Sink pushes Kafka data to external systems.
- **spark.mllib vs spark.ml**: Old (RDD) vs new (DataFrame); modern code uses spark.ml with Pipelines.
- **Transformer vs Estimator**: Transformer outputs a DataFrame directly; Estimator learns from data, produces a Transformer.

Sample Exam Questions (CCEE Level)

Q1: Which of the following is NOT a step in data preprocessing?

- a) Data cleaning
- b) Data visualization
- c) Data integration
- d) Data transformation
  Ans: b) Data visualization (that is EDA, not preprocessing)

Q2: In Kafka, a consumer group helps with:

- a) Parallelism by sharing partitions across consumers
- b) Fault tolerance by reassigning partitions when a consumer fails
- c) Both a and b
- d) Neither a nor b
  Ans: c) Both a and b

Q3: Spark SQL Catalyst is used for:

- a) Storing data on disk
- b) Query optimization (logical and physical plans)
- c) Writing messages to Kafka
- d) Creating Hive Metastore
  Ans: b) Query optimization

Q4: In Spark MLlib, a Transformer:

- a) Learns a model from data
- b) Takes a DataFrame and returns another (possibly with new columns)
- c) Manages cluster nodes
- d) Only works with RDDs
  Ans: b) Takes a DataFrame and returns another

Q5: Structured Streaming differs from classic Spark Streaming by using:

- a) DStream abstraction
- b) DataFrame/Dataset abstraction
- c) Only for batch processing
- d) MapReduce engine
  Ans: b) DataFrame/Dataset abstraction

Last Updated: Session 33 (Spark MLlib & Predictive Analysis)
CCEE Exam Status: Big Data Technologies Sessions 27–33 ✅ Covered
Total Sessions Covered: 7 (27–33)
Total Topics: 20+ core concepts with MCQ practice

Next Steps for Exam Preparation

- Review terminology and definitions multiple times.
- Practice more CCEE-level MCQs focusing on multi-select questions.
- Understand the relationship and differences between similar concepts (e.g., Preprocessing vs EDA, DStream vs DataFrame, Producer vs Consumer).
- Practice mini case studies: "Which technology would you use for X scenario?"
- Remember: This module integrates all prior Spark and Kafka modules; use this as a comprehensive reference.